

XSD: язык описания структуры XML-документов

Лекция по теории XML Schema с примерами из практической схемы

Август 2026

- [Введение](#)
- [1. XSD — это не язык программирования](#)
- [2. Простые и сложные типы](#)
 - [2.1. Простые типы \(`xs:simpleType` \)](#)
 - [2.2. Сложные типы \(`xs:complexType` \)](#)
- [3. Модельные группы: `sequence`, `choice`, `all`](#)
 - [3.1. `xs:sequence` — порядок обязателен](#)
 - [3.2. `xs:choice` — выбор одного варианта](#)
- [4. Наследование типов](#)
 - [4.1. `Extension` — расширение базового типа](#)
 - [4.2. `Restriction` — сужение базового типа](#)
 - [4.3. Наследование простых типов](#)
- [5. Полиморфизм: `xsi:type` и `substitutionGroup`](#)
 - [5.1. `xsi:type`](#)
 - [5.2. `xs:choice` как альтернатива `xsi:type`](#)
 - [5.3. `substitutionGroup`](#)
 - [5.4. Сравнение трёх подходов](#)
- [6. Итоговая аналогия с языками программирования](#)
- [Заключение](#)

Введение

XSD (XML Schema Definition) — это язык описания структуры и содержимого XML-документов. Схема XSD задаёт, какие элементы и атрибуты допустимы в документе, в каком порядке они должны идти, какие у них типы данных и ограничения.

В качестве сквозного практического примера в этой лекции используется реальная промышленная схема — **ON_ZAKZVGO_UserContract** (схема заказа на перевозку груза, описывающая грузоотправителя, перевозчика, реквизиты организаций и маршрут). На её элементах разобраны все ключевые теоретические механизмы XSD.

1. XSD — это не язык программирования

Первое, что важно понять: несмотря на визуальное сходство с языками программирования (наличие типов, наследования, модульности), **XSD — декларативный язык**, а не императивный.

Он не выполняется, не вычисляет значения и не содержит логики ветвления в привычном смысле.

Полезная аналогия: XSD ближе к **системе типов**, вынесенной в отдельный документ (как `.d.ts` в TypeScript, Protocol Buffers `.proto` или JSON Schema), либо к **формальной грамматике** (в духе БНФ), которая описывает множество допустимых документов. Валидатор XSD не «исполняет» схему — он проверяет, принадлежит ли конкретный XML-документ этому множеству.

Есть в XSD	Нет в XSD
Система типов, наследование	Переменные, присваивание
Модульность (<code>import</code> , <code>include</code>)	Условная логика уровня «если поле A = X, то поле B обязательно»
Ограничения-инварианты (<code>pattern</code> , <code>enumeration</code>)	Циклы, функции
Композиция структур (<code>sequence</code> , <code>choice</code> , <code>all</code>)	Вычисляемые значения

Для условной бизнес-логики между полями (которую XSD принципиально не может выразить) поверх XSD применяют дополнительные инструменты — Schematron или XSLT.

2. Простые и сложные типы

2.1. Простые типы (`xs:simpleType`)

Простой тип описывает **текстовое содержимое** без атрибутов и дочерних элементов — число, строку, дату, перечисление и т.д. Простые типы всегда базируются на встроенных типах XSD (`xs:string` , `xs:decimal` , `xs:date` и др.) через **ограничение** (`xs:restriction`).

Пример из схемы — тип для ОГРН:

```
<xs:simpleType name="ogrn">
  <xs:annotation>
    <xs:documentation>ОГРН - ОГРНИП</xs:documentation>
  </xs:annotation>
  <xs:restriction base="xs:string">
    <xs:minLength value="13" />
    <xs:maxLength value="15" />
    <xs:pattern value="[0-9]{13,15}" />
  </xs:restriction>
</xs:simpleType>
```

Здесь встроенный тип `xs:string` сужается: значение обязано состоять из 13–15 цифр. 13 цифр — ОГРН юридического лица, 15 — ОГРНИП индивидуального предпринимателя.

2.2. Сложные типы (`xs:complexType`)

Сложный тип описывает элемент, который может содержать **дочерние элементы и/или атрибуты**. Именно сложные типы формируют структуру документа.

```
<xs:complexType name="DocumentDetails">
  <xs:attribute name="Type" type="string1000" use="required"/>
  <xs:attribute name="DocumentNumber" type="string100" use="required"/>
  <xs:attribute name="DocumentDate" type="date" use="required"/>
</xs:complexType>
```

3. Модельные группы: sequence, choice, all

Модельные группы определяют, **как компонуются** дочерние элементы внутри сложного типа.

Группа	Семантика
<code>xs:sequence</code>	Элементы обязаны идти строго в заданном порядке
<code>xs:all</code>	Все перечисленные элементы должны присутствовать, но порядок не важен (в XSD 1.0 — только с <code>minOccurs</code> / <code>maxOccurs</code> равными 0 или 1)
<code>xs:choice</code>	Из перечисленных вариантов должен быть выбран ровно один (или заданное число раз)

3.1. `xs:sequence` — порядок обязателен

Большинство узлов схемы построено на `sequence`. Например, фрагмент заказа:

```
<xs:element maxOccurs="unbounded" name="IntermediatePoint" type="string1000">
  <xs:annotation>
    <xs:documentation>ПромПунктМрш - Наименование промежуточного пункта маршрута</xs:documentation>
  </xs:annotation>
</xs:element>
```

3.2. `xs:choice` — выбор одного варианта

`xs:choice` моделирует ситуацию «либо одно, либо другое», по аналогии с `union` / `enum` вариантов в языках программирования. В разбираемой схеме этот паттерн используется многократно — для описания сведений об организации:

```
<xs:complexType name="ExtendedOrganizationInfoLogistics">
  <xs:choice>
    <xs:element name="OrganizationDetails" type="ExtendedOrganizationDetailsLogistics" />
    <xs:element name="OrganizationReference" type="ExtendedOrganizationReferenceLogistics" />
  </xs:choice>
</xs:complexType>
```

Смысл: сведения об организации (грузоотправителе, перевозчике и т.д.) либо вводятся **вручную целиком** (`OrganizationDetails` — ИНН, ОГРН, адрес), либо задаются **ссылкой** на уже зарегистрированную в системе организацию (`OrganizationReference` — по идентификатору `BoxId`). Оба варианта одновременно в документе недопустимы, как и отсутствие обоих.

Валидно:

```
<Shipper>
  <OrganizationDetails OrgName="000 «Ромашка»" Inn="7707083893" OrgType="LegalEntity"/>
</Shipper>
```

Невалидно (оба варианта сразу):

```
<Shipper>
  <OrganizationDetails .../>
  <OrganizationReference .../>
</Shipper>
```

По умолчанию `minOccurs="1" maxOccurs="1"`, но их можно менять — например, `maxOccurs="unbounded"` позволяет **повторять** выбор произвольное число раз в любом порядке (чередование вариантов), а `minOccurs="0"` делает весь выбор необязательным.

`xs:choice` может содержать не только отдельные элементы, но и вложенные группы (`sequence`, другой `choice`), формируя более сложные альтернативы.

4. Наследование типов

XSD поддерживает наследование через `xs:complexContent` (для сложных типов) и `xs:simpleContent` (когда сложный тип расширяет простой, добавляя только атрибуты). Внутри указывается либо `xs:extension`, либо `xs:restriction`.

4.1. Extension — расширение базового типа

Самый частый механизм: дочерний тип наследует все элементы/атрибуты базового и добавляет свои. В разбираемой схеме — цепочка типов для описания организации:

```
<xs:complexType name="ExtendedOrganizationDetails_ManualFilling_Logistics">
  <xs:sequence>
    <xs:element name="Phones">...</xs:element>
    <xs:element minOccurs="0" name="Emails">...</xs:element>
    <xs:element minOccurs="0" name="OrganizationAdditionalInfos">...</xs:element>
    <xs:element minOccurs="0" name="ForeignPersonDetails" type="DocumentDetails"/>
  </xs:sequence>
  <xs:attribute name="Ogrn" type="ogrn">
    <xs:annotation><xs:documentation>ОГРН - ОГРНИП</xs:documentation></xs:annotation>
  </xs:attribute>
  <xs:attribute name="OtherContactInfo" type="string255"/>
</xs:complexType>
```

```
<xs:complexType name="ExtendedOrganizationDetailsLogistics">
  <xs:complexContent>
    <xs:extension base="ExtendedOrganizationDetails_ManualFilling_Logistics">
      <xs:sequence>
        <xs:element minOccurs="0" name="Address" type="AddressInfo"/>
      </xs:sequence>
      <xs:attribute name="OrgName" type="string1000" use="required"/>
      <xs:attribute name="Inn" type="inn"/>
      <xs:attribute name="Kpp" type="kpp"/>
      <xs:attribute name="OrgType" type="OrganizationType_DatabaseOrder" use="required"/>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

```

<!-- ... -->
</xs:extension>
</xs:complexContent>
</xs:complexType>

```

ExtendedOrganizationDetailsLogistics наследует от базового типа атрибут `Ogrn` и элементы `Phones / Emails`, и добавляет собственные: `OrgName`, `Inn`, `Kpp`, `Address`, `OrgType`. Именно поэтому у элемента `Shipper` в итоговом документе доступен атрибут `Ogrn`, хотя напрямую в типе `ExtendedOrganizationDetailsLogistics` он не объявлен — он пришёл по цепочке наследования.

Важное правило: если базовый тип использует `sequence`, а расширение добавляет свои элементы тоже через `sequence`, то элементы расширения добавляются **после** элементов базового типа — порядок фиксирован.

Полная цепочка наследования для `Shipper`:

```

ExtendedOrganizationDetails_ManualFilling_Logistics  (база: Phones, Emails, Ogrn, OtherContactInfo)
| extension
▼
ExtendedOrganizationDetailsLogistics                (+ Address, OrgName, Inn, Kpp, OrgType)
| используется в choice
▼
ExtendedOrganizationInfoLogistics                    (choice: OrganizationDetails | OrganizationReference)
| тип элемента
▼
<Shipper type="ExtendedOrganizationInfoLogistics">

```

4.2. Restriction — сужение базового типа

Используется реже — например, для сужения диапазона `minOccurs/maxOccurs`, ограничения перечисления значений или превращения необязательного элемента в обязательный. В отличие от `extension`, `restriction` требует **заново прописать всю структуру** типа (не только отличия), но эта структура должна оставаться допустимым подмножеством базового типа.

4.3. Наследование простых типов

Для простых типов наследование возможно только через `restriction` (`extension` не применяется). Ровно так устроен разобранный ранее тип `ogrн : xs:string` сужается до строки из 13-15 цифр.

Есть и промежуточный случай — `xs:simpleContent`, когда сложный тип расширяет простой базовый тип, добавляя атрибуты:

```

<xs:complexType name="PriceType">
  <xs:simpleContent>
    <xs:extension base="xs:decimal">
      <xs:attribute name="currency" type="xs:string"/>
    </xs:extension>
  </xs:simpleContent>
</xs:complexType>

```

5. Полиморфизм: xsi:type и substitutionGroup

XSD поддерживает механизмы, аналогичные полиморфизму в ООП — когда элемент объявлен одним (базовым/абстрактным) типом, а по факту в документе используется производный.

5.1. xsi:type

Атрибут `xsi:type` (из пространства имён `http://www.w3.org/2001/XMLSchema-instance`) ставится **непосредственно в XML-документе** на конкретном элементе и явно указывает, каким производным типом его следует валидировать.

Схема:

```
<xs:complexType name="Animal" abstract="true">
  <xs:sequence>
    <xs:element name="Name" type="xs:string"/>
  </xs:sequence>
</xs:complexType>

<xs:complexType name="Dog">
  <xs:complexContent>
    <xs:extension base="Animal">
      <xs:sequence>
        <xs:element name="Breed" type="xs:string"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>

<xs:element name="Pet" type="Animal"/>
```

Документ:

```
<Pet xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:type="Dog">
  <Name>Рекс</Name>
  <Breed>Овчарка</Breed>
</Pet>
```

Тип `Animal` объявлен `abstract="true"` — использовать его напрямую нельзя, поэтому в документе обязательна подстановка конкретного производного типа через `xsi:type`. Если бы `Animal` не был абстрактным, `xsi:type` был бы не обязателен, а опциональным способом «подменить» тип элемента на более специфичный.

5.2. xs:choice как альтернатива xsi:type

Разбираемая схема ОСОЗНАННО избегает `xsi:type` для описания вариативности организации, используя вместо этого `xs:choice` с именованными элементами-вариантами (см. раздел 3.2). Это не единственно возможный, но частый выбор для российских государственных/ведомственных XSD-схем, по нескольким причинам:

1. Именованные варианты **более явные и читаемые** — тип виден сразу по имени тега, без необходимости заглядывать в объявления типов.
2. `xsi:type` требует объявления дополнительного пространства имён `xsi` в каждом документе — источник ошибок при ручной или legacy-генерации XML.
3. Не все инструменты и генераторы одинаково хорошо поддерживают `xsi:type`.

5.3. substitutionGroup

Третий механизм полиморфизма — позволяет одному элементу **заменять** другой элемент в документе (а не просто подменять тип), если они совместимы. В отличие от `xsi:type` (подмена типа) и `xs:choice` (явный список вариантов, известных заранее), `substitutionGroup` позволяет расширять список допустимых **элементов** без изменения исходной схемы — например, сторонние схемы могут декларировать новые элементы, замещающие базовый.

5.4. Сравнение трёх подходов

Механизм	Как выглядит в XML	Когда применяют
<code>xsi:type</code>	<code><Pet xsi:type="Dog"></code>	Много производных типов, схема не перечисляет их явно в месте использования
<code>xs:choice</code> с именованными элементами	<code><Shipper></code> <code><OrganizationDetails/></code> <code></Shipper></code>	Вариантов немного, важна явность по имени тега (типичный подход в госсхемах)
<code>substitutionGroup</code>	элемент <code>SpecialPet</code> замещает <code>Pet</code> целиком	Нужно расширять список допустимых элементов, не переписывая базовую схему

6. Итоговая аналогия с языками программирования

Концепция XSD	Аналог в языках программирования
<code>simpleType</code> / <code>complexType</code>	Классы, типы данных
<code>extension</code>	Наследование (<code>extends</code>)
<code>restriction</code>	Сужение типа, инварианты (branded types, assertions)
<code>sequence</code>	<code>struct</code> с фиксированным порядком полей
<code>choice</code>	<code>union</code> / <code>enum</code> вариантов
<code>all</code>	Объект с необязательными полями без порядка
<code>abstract="true"</code>	Абстрактный класс / интерфейс
<code>xsi:type</code>	Полиморфизм, приведение к производному типу в рантайме
<code>import</code> / <code>include</code>	Модули, пространства имён

При этом ключевое отличие остаётся неизменным: XSD **не исполняется** — он лишь декларативно описывает множество допустимых документов, а всю проверку выполняет внешний валидатор. Любая межполевая условная логика («если поле А равно X, то поле Б обязательно») выходит за пределы возможностей чистого XSD и требует Schematron или XSLT.

Заключение

На примере промышленной схемы `ON_ZAKZVG0_UserContract` (заказ на перевозку груза) мы разобрали основные механизмы XSD:

- разделение на простые и сложные типы;
- три модельные группы компоновки элементов (`sequence` , `choice` , `all`);
- два механизма наследования типов (`extension` , `restriction`);
- полиморфизм через `xsi:type` , `xs:choice` и `substitutionGroup` .

Эти механизмы дают XSD выразительность, сопоставимую с системой типов языка программирования, но не превращают его в язык программирования: XSD остаётся декларативным инструментом описания структуры данных, а не вычислений.